# Lesson 3 · Rebuild RAG with LangChain

**PDF:** this lesson · **Install (Linux · macOS · Windows):** guide · PDF

> **Part of local-ai-lab** — a hands-on course for building local AI.

> **Course home:** https://nikolareljin.github.io/local-ai-lab/ **Source:** https://github.com/nikolareljin/local-ai-lab

> **Lessons:** 1 · RAG → 2 · MCP → **3 · LangChain (you are here)** → 4 · LangGraph → 5 · Ollama tools → 6 · Semantic Kernel → 7 · Bedrock Agents → 8 · Google ADK

> **Status: planned.** Outline below; full step-by-step coming later. ⭐ the repo to follow along.

---

## The idea

In Lesson 1 you built every RAG primitive by hand: a loader, a splitter, a retriever, a prompt, and a provider abstraction. **LangChain** ships all of those as components. This lesson rebuilds the **same** document Q&A app with LangChain — and then asks the honest question: **what did the framework buy us, and what did it cost?**

The goal is not to crown a winner. It's that, having written the primitives yourself, you can now read LangChain and see **exactly** which of your hand-rolled pieces each component replaces.

## What you'll learn

- **Document loaders & text splitters** — LangChain's `PyPDFLoader`, `RecursiveCharacterTextSplitter` vs. your `extract.py` / `chunk.py`
- **Vector stores & retrievers** — `Chroma` / `FAISS` + `as_retriever()` vs. your BM25/embeddings
- **Chat models & embeddings** — `ChatOllama`, `ChatOpenAI`, `init_chat_model` vs. your provider adapters (the abstraction you already understand)
- **LCEL** — composing a retrieval chain with the LangChain Expression Language (`|` pipes)
- **Prompt templates** — `ChatPromptTemplate` vs. your `prompts.py`
- **Trade-offs** — abstraction and ecosystem vs. transparency, dependencies, and version churn

## Maps directly onto Lesson 1

| You built by hand (Lesson 1) | LangChain component (Lesson 3) |
|---|---|
| `extract.py` | `DocumentLoaders` (`PyPDFLoader`, `Docx2txtLoader`, …) |
| `chunk.py` | `RecursiveCharacterTextSplitter` |

| | |
|---|---|
| `store.py` index | `VectorStore` (`Chroma`, `FAISS`) |
| `Bm25Retriever` / `EmbeddingRetriever` | `BM25Retriever`, `vectorstore.as_retriever()` |
| `prompts.py` | `ChatPromptTemplate` |
| `providers/` | `ChatModel` adapters (`ChatOllama`, `ChatOpenAI`, …) |
| `engine.answer_question()` | an LCEL retrieval chain |

> **The payoff of doing it the hard way first:** none of the table above is mysterious. You know what every component does because you wrote its equivalent.

## Prerequisites

Finish Lesson 1. Bring the same `documents/` corpus — we'll get the same answers, built a different way.

## Next lesson

**Lesson 4 · LangGraph →** — turn the chain into a stateful agent graph.

---